XDL Demo 指南

基于 RPP 加速卡

XDL Technical Support

2026-03-19

文档目的

本文档是 **RPP Demo Guide** 的官方客户交付 **SOP**，面向 终端客户、系统集成商和解决方案合作伙伴，用于在 **RPP AI** 加速硬件 上 独立完成 **Demo** 应用的执行、验证与故障排查。

本指南重点关注 可落地的执行步骤、实时验证以及首次运行成功率，并体现了 经 **XDL** 技术支持团队验证的推荐 **Demo** 工作流程。

目录

- 1. 驱动安装与验证
  - 1.1 验证 AzureBlade 设备连接
  - 1.2 安装 RPP 内核驱动
  - 1.3 驱动安装故障排查
- 2. 运行时服务
  - 2.1 RPP Server
  - 2.2 AE-SMI 监控工具
- 3. LLM Demo
  - 3.1 执行前准备
  - 3.2 Llama3 Demo
  - 3.3 Qwen3 Demo
  - 3.4 Qwen2.5 Demo
  - 3.5 Deepseek Demo
- 4. 重启 RPP Server
- 5. OpenRT 示例
  - [5.1 OpenRT（XDL CV 推理）模型支持矩阵]
    - 5.1.1 支持的图像分类模型
    - 5.1.2 支持的目标检测模型

  - 5.1.3 说明
- [5.2 C++ Demo]
  - 5.2.1 YOLOv8【目标检测】
  - 5.2.2 ResNet50【分类】
  - 5.2.3 ResNet50 多线程【并行处理】
- [5.3 Python Demo]
  - [5.3.1 YOLOv8【Python Demo – 目标检测】]
  - [5.3.2 YOLOv5【Python Demo – 目标检测】]
  - [5.3.3 ResNet50【Python Demo – 图像分类】]
  - [5.3.4 分类【Python Model Zoo Demo】]
- [5.4 C++ 沙盒]
  - [5.4.1 运行 OpenRT C++ 沙盒]
  - [5.4.2 基于 JSON 配置的 OpenRT 沙盒]
- [5.5 Python 沙盒]
  - [5.5.1 OpenRT Python 沙盒]

- 6. 高级应用
  - 6.1 OpenStream 应用（启用视频解码）
- [7. CUDA 示例]
- 8. YOLOv3 压力测试示例
  - 8.1 运行 YOLOv3 压力测试
  - 8.2 基于循环的压力测试（推荐）
  - [8.3 命令行选项]
  - 8.4 示例命令
  - [8.5 说明与最佳实践]
- 9. 已知问题与故障排查
  - [9.1 常见问题]
  - [9.2 排查步骤]
- 10. 结论
- 11. 致谢
- 12. 反馈与贡献
- 13. 免责声明
- 14. 版权声明
- 15. 技术支持

- 16. 联系方式
- 17. 法律声明
- [18. 商标说明]
- 19. 注释
- 20. 安全与合规
- 21. 发布说明
- 修订记录

1. 驱动安装与验证

目标：验证 RPP 硬件连接状态，并安装 / 验证内核驱动以支持 Demo 运行。

1.1 验证 AzureBlade 设备连接

为确认 AzureBlade 加速卡已正确连接并枚举在 PCIe 总线上，请执行以下命令：

```
lspci | grep Co-processor
```

期望输出：

```
XX:XX.X Co-processor: Device 1f2e:00a1 (rev 01)
```

说明：XX:XX.X 为 BDF 信息，例如 **05:00.0**（Bus 05，Device 00，Function 0）。

若存在一个或多个 RPP 加速卡以 Co-processor 形式列出，则说明设备已被正确识别。

注意：若未检测到设备，请检查 PCIe 卡是否插紧以及 BIOS 中的 PCIe 相关配置。

1.2 安装 RPP 内核驱动

操作说明：该步骤 在每次系统重启或断电后都需要重新执行。这是符合当前内核安全策略的预期行为。

进入驱动源码目录并安装内核模块：

```
cd $HOME/azurengine/rpp_drv
sudo insmod rpp.ko
sudo chmod 666 /dev/rpp0_entire_ctrl /dev/ve0_entire_ctrl
```

通过查看内核日志验证安装是否成功：

```
dmesg
```

若未出现错误信息，则表示驱动已成功加载，可以执行 Demo。

1.3 驱动安装故障排查

若驱动安装失败，请参考以下常见错误及解决方法。

错误 **1**：模块已加载
错误信息：

```
insmod: ERROR: could not insert module rpp.ko: File exists
```

解决方法：

该信息表示驱动已加载，无需任何额外操作。

错误 **2**：模块格式无效
错误信息：

```
insmod: ERROR: could not insert module rpp.ko: Invalid module format
```

解决方法：

针对当前内核版本重新编译驱动模块：

```
cd $HOME/azurengine/rpp_drv
make clean
make -j$(nproc)
```

随后重新执行 **1.2** 节中的安装步骤。

2. 运行时服务

目标：启动并管理 Demo 运行所需的 RPP 运行时服务。

2.1 RPP Server

RPP Server 为多进程访问 RPP 硬件资源提供运行时服务，是所有 基于 **OpenRT** 的 **Demo** 以及多应用场景的必备组件。

启动 **RPP Server**：

```
cd /usr/local/rpp/bin
./rpp_server
```

> 重要：在运行依赖的 Demo（OpenRT、OpenStream、DeepSort、CUDA 示例）期间，请保持该终端窗口处于打开状态。
>
> 例外：**LLM Demo** 运行时必须关闭 **RPP Server**。

2.2 AE-SMI 监控工具

AE-SMI 工具用于实时监控 RPP 设备状态，包括：

- 固件版本
- PCIe 状态
- DDR 配置
- 核心频率
- 温度
- 利用率
- 功耗

启动命令：

```
cd /usr/local/rpp/bin/
./ae-smi
```

按 **q** 键退出 ae-smi 工具。

3. LLM Demo

> 目标：在无外部网络连接的情况下，在设备端原生运行大语言模型（Llama 系列、Qwen 系列、Deepseek 系列等）。

RPP 支持以 Python 推理脚本形式原生运行大语言模型，并针对设备端执行进行了优化。

3.1 执行前准备

> 重要：在运行任何 LLM Demo 之前，必须确保 **RPP Server** 未在运行。

确认系统中不存在 rpp_server 进程：

```
# 第一个命令用于查询正在运行的 rpp_server 进程并获取 PID
ps -ef | grep rpp_server
# 第二个命令用于杀掉对应 PID 的 rpp_server 进程
sudo kill -9 <PID>
# 第三个命令用于确认 rpp_server 已完全退出
ps -ef | grep rpp_server
```

3.2 Llama3 Demo

步骤 1：下载模型
下载并解压 Llama3 模型（请从 XDL 官方发布渠道获取最新模型）：

```
# 下载模型并解压到你使用的路径
```

步骤 2：运行推理

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/llama3/llama3_graph_demo/
python llama3_run_graph_8k_dyn_v1_6_0_6.py -g "模型 graph 目录路径"
```

可选参数：

```
-h, --help                    显示帮助信息并退出
-g GRAPH_PATH, --graph_path GRAPH_PATH
                              模型 graph 目录路径
-r RUN_MODE, --run_mode RUN_MODE
                              执行模式
```

示例：

```
# 以 $HOME/model_zoo/llm/llama3.1_8b_graph 为例
python llama3_run_graph_8k_dyn_v1_6_0_6.py -g $HOME/model_zoo/llm/
llama3.1_8b_graph
# 输入提示词，例如："introduce yourself"
# 使用 Ctrl+C 结束运行
```

步骤 3：退出环境与后处理

```
conda deactivate
```

3.3 Qwen3 Demo

步骤 1：下载模型
下载并解压 Qwen3 模型（请从 XDL 官方发布渠道获取最新模型）：

```
# 下载模型并解压到你使用的路径
```

步骤 2：运行推理

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/qwen3/qwen3_graph_demo/
python qwen3_run_graph_8k_v1_6_0_8.py -g "模型 graph 目录路径"
```

可选参数：

```
-h, --help                    显示帮助信息并退出
-g GRAPH_PATH, --graph_path GRAPH_PATH
                              模型 graph 目录路径
-r RUN_MODE, --run_mode RUN_MODE
                              执行模式
```

示例：

```
python qwen3_run_graph_8k_v1_6_0_8.py -g $HOME/model_zoo/llm/qwen3_8b_graph
# 输入提示词，例如："introduce yourself"
# 使用 Ctrl+C 结束运行
```

步骤 3：退出环境与后处理

```
conda deactivate
```

3.4 Qwen2.5 Demo

步骤 1：下载模型
下载并解压 Qwen2.5 模型（请从 XDL 官方发布渠道获取最新模型）：

```
# 下载模型并解压到你使用的路径
```

步骤 2：运行推理

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/qwen2.5/qwen2p5_graph_demo
python qwen2p5_run_graph_8k_v1_6_0_8.py -g "模型 graph 目录路径"
```

可选参数：

```
-h, --help                    显示帮助信息并退出
-g GRAPH_PATH, --graph_path GRAPH_PATH
                              模型 graph 目录路径
```

```
-r RUN_MODE, --run_mode RUN_MODE
                              执行模式
```

示例：

```
python qwen2p5_run_graph_8k_v1_6_0_8.py -g $HOME/model_zoo/llm/
qwen2p5_7b_graph
# 输入提示词，例如："introduce yourself"
# 使用 Ctrl+C 结束运行
```

步骤 3：退出环境与后处理

```
conda deactivate
```

3.5 Deepseek Demo

步骤 1：下载模型
下载并解压 Deepseek 模型（请从 XDL 官方发布渠道获取最新模型）：

```
# 下载模型并解压到你使用的路径
```

步骤 2：运行推理

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/deepseek_series/DeepSeek-R1-Distill-Qwen2p5/graph_demo
python deepseek_qwen_7b_run_graph_8k_v1_6_0_8.py -g "模型 graph 目录路径"
```

可选参数：

```
-h, --help                    显示帮助信息并退出
-g GRAPH_PATH, --graph_path GRAPH_PATH
                              模型 graph 目录路径
-r RUN_MODE, --run_mode RUN_MODE
                              执行模式
```

示例：

```
python deepseek_qwen_7b_run_graph_8k_v1_6_0_8.py -g $HOME/model_zoo/llm/
deepseek_r1_qwen_7b_graph
# 输入提示词，例如："introduce yourself"
# 使用 Ctrl+C 结束运行
```

步骤 **3**：退出环境与后处理

```
conda deactivate
```

4. 重启 RPP Server

完成 LLM Demo 后，请重新启动 RPP Server，以便后续运行 OpenRT 或视频流水线相关 Demo：

```
cd /usr/local/rpp/bin
./rpp_server
```

5. OpenRT 示例

目标：通过优化后的模型与实时验证，验证高性能推理执行能力。

所有 OpenRT 示例都要求 **RPP Server** 已启动。服务器启动方式请参考第 4 章。

5.1 OpenRT 模型支持矩阵（XDL CV 推理）

本节总结了 **XDL** 在 **OpenRT** 推理中官方支持的计算机视觉（**CV**）模型，涵盖 图像分类 与 目标检测 两类工作负载。

5.1.1 支持的图像分类模型

以下分类模型已在 XDL 后端完成验证，可直接用于 OpenRT 推理：

模型名称	模型名称	模型名称
BVLC AlexNet	CaffeNet	DenseNet
GoogLeNet	Inception	MobileNet
ResNet-101	ResNet-152	ShuffleNet
SqueezeNet	VGG19	VGG19 (Caffe2)
YOLOv8-CLS	ZFNet-512	YOLOv11-CLS

5.1.2 支持的目标检测模型

以下目标检测模型支持在 OpenRT 上进行实时推理：

模型名称
YOLOv5
YOLOv8
YOLOv11

5.1.3 说明

- 所列模型均已在 OpenRT 上通过 **XDL Runtime** 的功能验证。
- 实际性能与支持特性可能因 模型配置、输入分辨率 及 硬件平台 不同而有所差异。
- 对于自定义模型或未支持的网络结构，请联系 XDL 技术支持团队进行可行性评估。

5.2 C++ 示例

5.2.1 YOLOv8【目标检测】

YOLOv8 示例基于 **OpenRT** 推理引擎，使用 YOLO 模型进行实时目标检测。

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/yolo
./yolov8m_demo
```

命令行参数

YOLOv8 示例提供灵活的命令行参数，允许用户指定自定义模型、输入图像以及推理行为。

```
./yolov8m_demo -h
```

用法

```
Usage: Yolov8m demo [--help] [--onnx VAR] [--image VAR] [--verbose] [--loop
VAR]
```

可选参数

参数	说明
`-h, --help`	显示帮助信息并退出
`-o, --onnx <model.onnx>`	YOLO ONNX 模型文件路径默认：`yolov8m.onnx`
`-i, --image <image_path>`	用于推理的输入图像路径若未指定，则使用内置测试流程
`-v, --verbose`	启用详细日志，用于调试与性能分析
`--loop <count>`	推理循环执行次数默认：`1`

使用自定义模型进行推理

用户可将默认的 YOLOv8m 模型替换为 自定义 **ONNX** 模型（需与 OpenRT 兼容）：

```
./yolov8m_demo \
  --onnx /path/to/custom_yolov8.onnx
```

说明：

- 模型需符合 YOLOv8 的 ONNX 导出格式。

- 输入张量布局、预处理和输出解码需与 YOLOv8 规范一致。

使用自定义图像进行推理

```
./yolov8m_demo \
  --image /path/to/test_image.jpg
```

也可同时指定自定义模型与图像：

```
./yolov8m_demo \
  --onnx /path/to/custom_yolov8.onnx \
  --image /path/to/test_image.jpg
```

循环推理（性能评估）
`--loop` 参数可用于多次推理执行，适合 性能基准测试：

```
./yolov8m_demo \
  --loop 100
```

该命令将执行 **100** 次推理，用于测量平均时延与吞吐率。

详细日志模式
启用详细日志以输出以下信息：

- 模型加载状态
- 张量初始化过程
- 推理执行细节

```
./yolov8m_demo --verbose
```

5.2.2 ResNet50【分类】

ResNet-50 示例基于 OpenRT 引擎，提供 高精度图像分类推理。 默认支持 FP32/BF16 执行，并可选启用 INT8 量化推理以提升性能并降低内存占用。

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/resnet
./resnet50_demo
```

命令行参数
ResNet-50 示例 仅支持 **ResNet** 系列模型，并支持灵活的运行时配置。

```
./resnet50_demo -h
```

用法

```
Usage: resnet50 demo. Supports ResNet series models only.
       [--help] [--onnx VAR] [--image VAR] [--verbose] [--loop VAR] [--int8]
```

可选参数

参数	说明
-h, --help	显示帮助信息并退出
-o, --onnx <model.onnx>	ResNet ONNX 模型文件路径默认：resnet50.onnx
-i, --image <image_path>	用于分类的输入图像路径若未指定，则使用默认测试图像
-v, --verbose	启用详细日志，用于调试和运行时检查
--loop <count>	推理循环次数默认：1

默认（高精度）推理
默认情况下，示例以 高精度模式 运行（根据平台支持使用 BF16）：

```
./resnet50_demo
```

也可指定自定义 ResNet ONNX 模型：

```
./resnet50_demo \
  --onnx /path/to/custom_resnet50.onnx
```

使用自定义图像

```
./resnet50_demo \
  --image /path/to/test_image.jpg
```

循环推理（性能评估）

```
./resnet50_demo \
  --loop 100
```

详细日志模式

```
./resnet50_demo --verbose
```

5.2.3 ResNet50 多线程【并行处理】

ResNet50 多线程示例展示了如何通过多线程处理 5,000 张图像，以减少数据传输时间并提升推理效率。

单线程执行：

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/resnet50_multi_thread
./resnet50_multi_thread
```

多线程执行（以 3 线程为例）：

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/resnet50_multi_thread
./resnet50_multi_thread -t=3
```

5.3 Python 示例

激活 Conda 环境：

```
conda deactivate
conda activate az_samples_env
```

5.3.1 YOLOv8【Python 示例 – 目标检测】

YOLOv8 Python 示例基于 **OpenRT** 推理引擎 实现实时目标检测，支持模型选择、图优化控制以及检测阈值调节等灵活运行选项。

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python yolov8.py
```

命令行参数

```
python yolov8.py -h
```

用法

```
usage: yolov8.py [-h] [-o ONNX] [-i IMAGE] [-v]
                 [--disable-dfs] [--disable-swish-fuse]
                 [--conf CONF] [--iou IOU]
```

可选参数

参数	说明
`-h, --help`	显示帮助信息并退出
`-o, --onnx <model.onnx>`	用于推理的 YOLOv8 ONNX 模型路径
`-i, --image <image_path>`	用于目标检测的输入图像路径
`-v, --verbose`	启用详细日志，用于调试和性能分析
`--disable-dfs`	禁用 **DFS**（动态融合调度）优化（默认启用）

参数	说明
`--disable-swish-fuse`	禁用 **Swish** 算子融合 优化（默认启用）
`--conf <value>`	置信度阈值，用于过滤检测结果
`--iou <value>`	NMS 的 IOU（交并比）阈值

默认参数运行

```
python yolov8.py
```

使用自定义模型

```
python yolov8.py \
  --onnx /path/to/custom_yolov8.onnx
```

说明：

- ONNX 模型需由 YOLOv8 导出并与 OpenRT 兼容。
- 输入分辨率与预处理需与模型定义一致。

使用自定义图像

```
python yolov8.py \
  --image /path/to/test_image.jpg
```

调整检测阈值

```
python yolov8.py \
  --conf 0.25 \
  --iou 0.45
```

- **Confidence (`--conf`)**：过滤低置信度目标
- **IOU (`--iou`)**：控制 NMS 抑制强度

控制 **OpenRT** 图优化

- **DFS**：关注功耗与稳定性
- **Swish Fuse**：关注性能
- 两者默认启用，可按需关闭

```
python yolov8.py --disable-dfs
```

```
python yolov8.py --disable-swish-fuse
```

详细日志模式

```
python yolov8.py --verbose
```

5.3.2 YOLOv5【Python 示例 – 目标检测】

YOLOv5 Python 示例基于 **OpenRT** 推理引擎 实现实时目标检测。
同时还提供可选的 **OpenCV DNN** 后端，用于功能对比与验证。

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python yolov5.py
```

命令行参数
查看所有支持的参数：

```
python yolov5.py -h
```

用法

```
usage: yolov5.py [-h] [-o ONNX] [-i IMAGE] [--dnn] [-v]
```

可选参数

参数	说明
`-h, --help`	显示帮助信息并退出
`-o, --onnx <model.onnx>`	用于推理的 YOLOv5 ONNX 模型路径
`-i, --image <image_path>`	用于目标检测的输入图像路径
`--dnn`	使用 **OpenCV DNN** 后端进行推理，而非 OpenRT
`-v, --verbose`	启用详细日志，用于调试与运行时检查

使用默认设置运行推理（**OpenRT**）

```
python yolov5.py
```

使用默认的 YOLOv5 ONNX 模型，并基于 OpenRT 后端执行目标检测。

使用自定义模型运行推理
用户可指定自定义的 YOLOv5 ONNX 模型：

```
python yolov5.py \
  --onnx /path/to/custom_yolov5.onnx
```

说明：

- ONNX 模型需由 YOLOv5 导出，并与 OpenRT 兼容。
- 输入分辨率与预处理方式需与模型定义保持一致。

使用自定义图像运行推理

```
python yolov5.py \
  --image /path/to/test_image.jpg
```

也可与自定义模型组合使用：

```
python yolov5.py \
  --onnx /path/to/custom_yolov5.onnx \
  --image /path/to/test_image.jpg
```

使用 **OpenCV DNN** 后端（可选）
为进行功能对比或调试，用户可使用 OpenCV DNN 后端运行推理：

```
python yolov5.py --dnn
```

典型使用场景：

- 验证模型正确性
- 对比 OpenRT 与 OpenCV DNN 的推理结果
- 调试预处理或后处理逻辑

详细日志模式
启用详细的运行时日志：

```
python yolov5.py --verbose
```

5.3.3 ResNet50【Python 示例 – 图像分类】

ResNet50 Python 示例基于 OpenRT 引擎，实现 高精度图像分类推理。 支持 ResNet-50 系列 ONNX 模型，并提供简洁的运行时配置选项。

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python resnet.py
```

命令行参数
查看所有可用参数：

```
python resnet.py -h
```

用法

```
usage: resnet.py [-h] [-o ONNX] [-i IMAGE] [-v]

resnet50 demo. Supports ResNet50 series models.
```

可选参数

参数	说明
-h, --help	显示帮助信息并退出
-o, --onnx <model.onnx>	用于分类的 ResNet50 ONNX 模型路径
-i, --image <image_path>	用于分类的输入图像路径
-v, --verbose	启用详细日志，用于调试与运行时检查

使用默认设置运行推理

```python
python resnet.py
```

使用默认的 ResNet50 ONNX 模型和内置测试图像执行分类推理。

使用自定义模型运行推理
用户可指定自定义的 ResNet50 ONNX 模型：

```python
python resnet.py \
  --onnx /path/to/custom_resnet50.onnx
```

说明：

- 仅支持 ResNet50 系列模型。
- ONNX 模型需遵循标准 ResNet50 的输入/输出定义。

使用自定义图像运行推理

```python
python resnet.py \
  --image /path/to/test_image.jpg
```

也可与自定义模型组合使用：

```python
python resnet.py \
  --onnx /path/to/custom_resnet50.onnx \
  --image /path/to/test_image.jpg
```

详细日志模式
启用详细的运行时日志，包括模型加载与推理执行信息：

```
python resnet.py --verbose
```

5.3.4 Classification【Python 模型仓库示例】

Classification 示例 允许用户对 OpenRT 模型仓库中 已支持的各类图像分类模型 进行单独评估。 主要用于分类网络的功能验证、精度检查及基础性能验证。

运行 **Classification** 示例

示例（以 GoogleNet 为例）:

```
cd $HOME/azurengine/openrt_samples/demos/python_demos/
run_classification_models
python run_classification_models.py \
  --onnx ../../../onnx_model_zoo/classification/googlenet-12.onnx \
  --image ../../../onnx_model_zoo/image_for_testing/liner.JPEG
```

命令行参数

查看所有支持的参数:

```
python run_classification_models.py -h
```

用法

```
usage: run_classification_models.py [-h] [-i IMAGE] [-o ONNX]
```

可选参数

参数	说明
`-h, --help`	显示帮助信息并退出
`-i, --image <image_path>`	用于分类的单张输入图像路径
`-o, --onnx <model.onnx>`	ONNX 分类模型路径

支持的使用场景

- 验证 模型仓库中分类网络 的功能正确性
- 对比不同网络结构的推理行为
- 验证 ONNX 模型与 OpenRT 的兼容性
- 使用单张测试图像进行快速一致性检查

说明

- 该示例 一次仅处理一张图像。
- ONNX 模型必须是 受支持的分类网络结构（如 AlexNet、GoogleNet、ResNet、VGG 等）。
- 输入预处理（尺寸调整、归一化）遵循模型的标准配置。

5.4 C++ Sandboxes（沙盒）

OpenRT C++ Sandbox 提供了灵活的验证环境，用于：

- 测试所有 官方支持的 **ONNX** 模型
- 验证 用户自定义 / 定制模型 的算子覆盖率与兼容性
- 使用 **MSE**（均方误差） 指标，与 **ONNX Runtime**（**ONNXRT**） 进行 数值精度对比

该沙盒主要用于 模型初始导入、功能验证以及产品集成前的精度验证。

5.4.1 运行 OpenRT C++ Sandbox

示例（以 ShuffleNet 为例）：

```
cd $HOME/azurengine/openrt_samples/build/sandboxes/openrt_cpp_sandbox
./openrt_cpp_sandbox \
  --onnx ../../../onnx_model_zoo/classification/shufflenet-v2-12.onnx
```

命令行参数

查看所有支持的参数：

```
./openrt_cpp_sandbox -h
```

用法

```
Usage: openrt c plus plus sandbox
       [--help] [--onnx VAR] [--verbose] [--warm_up]
```

可选参数

参数	说明
`-h, --help`	显示帮助信息并退出
`-o, --onnx <model.onnx>`	ONNX 模型文件路径默认：`data/model/resnet50.onnx`
`-v, --verbose`	启用详细日志，输出完整的运行时信息
`-w, --warm_up`	启用预热执行，用于在性能和精度测试中排除首次运行的初始化开销

与 **ONNX Runtime** 的精度验证

运行沙盒时：

- 同一 ONNX 模型将分别通过 **OpenRT** 与 **ONNX Runtime** 执行
- 对输出张量进行数值对比
- 计算 **MSE**（均方误差），用于评估推理结果的一致性

用户可借此：

- 验证算子实现的正确性
- 发现由优化或硬件加速引入的数值偏差
- 确保与参考 ONNX Runtime 执行结果在功能上的一致性

典型使用场景

- 验证 新模型或自定义 **ONNX** 模型 的算子支持情况
- 调试模型转换或图优化问题
- 执行 精度回归测试
- 在部署前评估 OpenRT 的可用性

说明

- 建议开启预热模式以获得稳定的性能测量结果。
- MSE 阈值会随模型结构和数据精度不同而变化。
- 对于 INT8 或 BF16 模型，出现微小数值差异属于正常现象。

5.4.2 支持 JSON 配置的 OpenRT Sandbox

OpenRT C++ Sandbox（JSON 配置模式）通过 JSON 配置文件支持 高级且完全可配置的测试场景。 该模式适用于 自动化测试、精度验证、引擎缓存 以及 **ONNX Runtime** 对比，无需频繁修改命令行参数。

示例（以 ResNet50 为例）:

```
cd $HOME/azurengine/openrt_samples/build/sandboxes/
openrt_cpp_json_config_sandbox
./openrt_cpp_json_config_sandbox --config=./configs/config.json
```

命令行参数

查看用法说明:

```
./openrt_cpp_json_config_sandbox -h
```

用法

```
Usage: openrt c plus plus sandbox
       [--help] [--config VAR] [--verbose]
```

可选参数

参数	说明
-h, --help	显示帮助信息并退出
-c, --config <config.json>	JSON 配置文件路径默认: config.json
-v, --verbose	启用详细日志，输出完整的运行时诊断信息

JSON 配置文件说明

以下为 JSON 配置示例及字段说明：

```
{
  "onnx_model_path" : "data/resnet50/resnet50.onnx",
  "log_file_path" : "log/log.txt",
  "openrt_engine_file_path" : "cached_engines/resnet50.engine",

  "inference_asynchronously" : false,
  "do_inference" : true,
  "inference_count" : 1,
  "rebuild_engine" : true,

  "__input_data_comment__": "use random data when 'image_file_path' and
'preprocessed_file_path' are not set.",
  "image_file_path" : "",
  "preprocessed_file_path" : "",
  "compare_with_onnx_runtime" : true,

  "model_output_folder" : "model_outputs"
}
```

- 模型与引擎配置

字段	说明
`onnx_model_path`	用于推理的 ONNX 模型路径
`openrt_engine_file_path`	用于缓存的 OpenRT 引擎序列化文件路径
`rebuild_engine`	若为 `true`，即使存在缓存引擎也会强制重新构建

- 日志与输出

字段	说明
`log_file_path`	运行时日志输出文件路径
`model_output_folder`	模型输出张量的存储目录

- 推理控制

字段	说明
`do_inference`	是否启用推理执行
`inference_count`	推理执行次数
`inference_asynchronously`	是否启用异步推理

- 输入数据配置

字段	说明
`image_file_path`	原始输入图像文件路径
`preprocessed_file_path`	预处理后的输入张量数据路径
(输入路径为空)	若两个字段均为空，则使用随机输入数据

- 精度验证

字段	说明
`compare_with_onnx_runtime`	是否启用与 ONNX Runtime 的数值对比
(对比方式)	对输出张量进行比较，并输出精度指标（如 MSE）

典型使用场景
- 自动化 模型回归测试
- 验证复杂或自定义模型的 算子支持情况
- 与 ONNX Runtime 进行 精度验证
- 引擎缓存与重建流程验证
- 同步或异步推理的性能评估

说明
- JSON 配置模式非常适合脚本化和 CI 环境使用。
- 在与 ONNX Runtime 对比时，BF16 / INT8 模型出现微小数值差异属于正常现象。
- 使用缓存引擎可显著降低多次运行时的启动延迟。

5.5 Python Sandboxes（沙盒）

激活环境：

```
conda deactivate
conda activate az_samples_env
```

5.5.1 OpenRT Python Sandbox

OpenRT Python Sandbox 提供了一个 轻量且灵活的验证环境，用于快速模型验证与调试。主要应用于 模型初期导入与功能验证阶段，在深入的 C++ 或系统级集成之前使用。

核心能力
- ✅ 测试所有 官方支持的 **ONNX** 模型
- ✅ 验证 用户自定义 / 定制 **ONNX** 模型 的算子覆盖率与兼容性
- ✅ 与 **ONNX Runtime**（**ONNXRT**） 进行 数值精度对比

  - 精度评估采用 **MSE**（均方误差）
- ✅ 基于 Python 的快速迭代、可视化日志与调试能力

推荐使用场景：

- 模型可行性评估
- 算子支持验证
- 部署前的精度一致性检查

运行 **Python Sandbox**
示例（使用 `yolov8m.onnx`）：

```
cd $HOME/azurengine/openrt_samples/sandboxes/openrt_python_sandbox
python openrt_python_sandbox.py -o ../../onnx_model_zoo/object_detection/
yolov8m.onnx
```

命令行参数
查看用法说明：

```
python openrt_python_sandbox.py -h
```

用法

```
usage: openrt python sandbox [-h] [-o ONNX] [-v]
```

可选参数

参数	说明
`-h, --help`	显示帮助信息并退出
`-o, --onnx <model.onnx>`	ONNX 模型文件路径
`-v, --verbose`	启用详细日志，输出完整的执行过程

典型工作流程
1. 加载目标 ONNX 模型
2. 使用 OpenRT 后端执行推理
3. 使用 ONNX Runtime 执行同样的推理
4. 对比输出张量并报告 **MSE** 指标
5. 通过日志定位不支持的算子或数值偏差

说明
- Python Sandbox 适合 快速验证，不适用于极限性能测试。
- 以下情况下出现微小数值差异属于正常现象：

  - 量化模型（INT8 / INT4）
  - 低精度数据类型（BF16）
- 如需性能分析或大规模测试，建议使用 **C++ Sandbox**。

6. 高级应用

> 目标：基于 **OpenRT** 与 **OpenStream**，构建并运行 实时、多路视频分析流水线，充分利用 硬件加速的视频解码、推理与目标跟踪能力。

6.1 OpenStream 应用（启用视频解码）

OpenStream 是一款高性能视频分析框架，面向 实时多路视频推理 场景设计。支持 最多 **16** 路并发视频流，并提供硬件加速的视频解码与 AI 推理能力。

典型应用场景包括：

- 多摄像头安防监控
- 实时目标检测与跟踪
- 边缘 AI 视频分析流水线

6.1.1 运行 OpenStream（YOLOv5 示例）

启动 OpenStream YOLOv5 推理应用：

```
cd $HOME/azurengine/openstream/build/app/openstream-infer-yolov5/
./openstream-infer-yolov5 ./configs/config_app.yaml
```

停止应用的方法：

- 按 **Ctrl + C**，或
- 使用 `sudo kill -15 <pid>` 进行优雅退出

6.1.2 应用配置概览

OpenStream 采用 两级配置模型：

- **`config_app.yaml`** 控制日志级别与流水线整体结构
- **`config_pipeline.yaml`** 定义端到端的视频处理流水线

6.1.3 `config_app.yaml` 说明

```
# log-level 0: TRACE, 1: DEBUG, 2: INFO, 3: WARN, 4: ERROR, 5: CRITICAL, 6:
OFF
log_info:
    spdlog_level: 2
    gstlog_level: 4
    spdlog_flush_level: 4
    spdlog_flush_time: 1
```

```yaml
pipeline_info:
    pipeline_num: 1
    pipeline_config1: ./configs/config_pipeline.yaml
```

关键参数说明

字段	说明
spdlog_level	应用日志详细级别
gstlog_level	GStreamer 内部日志级别
pipeline_num	并行流水线数量
pipeline_config1	流水线配置文件路径

6.1.4 config_pipeline.yaml 说明

该文件定义了 完整的视频分析流水线，包括输入源、推理、跟踪、OSD 以及输出。

```yaml
common:
    show_image:  True
    perf_monitor: NONE   # ./configs/config_monitor.yaml
    replay: False
    cpu_decoder: False
```

通用配置项

参数	说明
show_image	是否显示推理结果
perf_monitor	性能监控配置
replay	是否回放视频流
cpu_decoder	使用 CPU 解码（默认关闭）

视频源配置

```yaml
source:
    factory: uridecodebin3
    video: file:///./video/sample_720p.mp4
```

- uridecodebin3 支持自动格式识别
- 支持 **RTSP**、**MP4**、**MKV** 等多种格式

推理阶段（**YOLOv5**）

```
infer1:
    factory: aeosinfer
    config_file: ./configs/config_fall_yolov5s.yaml
```

- 执行硬件加速的 YOLOv5 推理
- 模型相关参数在对应配置文件中定义

目标跟踪

```
tracker:
    factory: aeostracker
    config_file: ./configs/config_fall_tracker.yaml
```

- 实现跨帧的多目标跟踪
- 提升时间一致性与目标 ID 稳定性

屏幕叠加显示（**OSD**）

```
OSD:
    factory: aeososd
    config_file: ./configs/config_fall_osd.yaml
```

- 绘制目标框、标签及跟踪 ID
- 支持自定义颜色、字体与叠加样式

输出端（**Sink**）

```
output:
    factory: fakesink
    config_file: NONE
```

- `fakesink` 会丢弃输出帧（适用于性能测试）
- 可根据需求替换为显示或文件输出 Sink

6.1.5 自定义视频输入配置

用户可以将默认视频源替换为 **IP** 摄像头 或 本地视频文件。

进入流水线配置目录：

```
cd $HOME/azurengine/openstream/build/app/openstream-infer-yolov5/configs
```

编辑 `config_pipeline.yaml` 的 第 **9** 行。

IP 摄像头（**RTSP**）示例

摄像头信息：

- IP: 192.168.1.100
- 用户名: admin
- 密码: admin

```
video: rtsp://admin:admin@192.168.1.100:554
```

本地 **MP4 / MKV** 文件示例

视频路径： /home/azurengine/azurengine/openstream/build/app/openstream-infer-yolov5/video/704_x264_fight.mkv

```
video: file:///home/azurengine/azurengine/openstream/build/app/openstream-infer-yolov5/video/704_x264_fight.mkv
```

6.1.6 注意事项与最佳实践

- 默认启用 硬件解码（cpu_decoder: False）
- 为获得最大吞吐量:
  - 关闭 show_image
  - 使用 fakesink 作为输出
- 通过增加 pipeline_num 启用多条流水线
- 运行 多路视频（≥8 路） 时，确保系统带宽与供电能力充足

7. CUDA 示例程序

目标：验证在 RPP 硬件上原生执行 CUDA Kernel 的能力，以支持异构计算负载。

RPP 支持直接运行多种 CUDA Kernel（如 FFT、矩阵转置等），并具备完整的 GPU 加速与 Tensor Core 优化能力。

可用的 **CUDA** 示例可执行程序:

```
cd $HOME/azurengine/cuda_samples/rpp_demo/build

# FFT 示例
./rpp_fft_v4/simpleRPPFFT_v4

# 通用矩阵乘（GEMM）示例
./rpp_gemm/simpleGemm

# 矩阵转置示例
./rpp_transpose/rpp_transpose
```

```
# 向量加法示例
./vector_add/vector_add
```

每个示例都会在 RPP 硬件上执行对应的 CUDA Kernel，并进行性能验证与结果校验。

8. YOLOv3 压力测试示例

> 目标：提供一个 通用、长时间运行的推理压力测试工具，用于验证在持续 AI 负载下的系统稳定性、可靠性以及功耗/热特性。

YOLOv3 示例 主要设计为 持续循环推理基准测试，而非功能展示型 Demo。推荐用于 长时间浸泡测试（**Soak Test**）、老化测试（**Burn-in**） 以及 RPP 硬件与 OpenRT 运行时的可靠性验证。

所有基于 OpenRT 的 Demo（包括该 YOLOv3 示例）均要求 **RPP Server** 正在运行。RPP Server 的启动方式请参考 第 **4** 章。

主要应用场景

- 长时间 可靠性与稳定性测试
- 持续 功耗 / 热压力验证
- 客户侧 老化与烧机测试
- 驱动 / 固件升级后的回归测试

与功能型 Demo 不同，该示例专注于 可重复、可控的推理循环，适合无人值守的长时间运行。

8.1 运行 YOLOv3 压力测试

YOLOv3 Demo 基于 **OpenRT** 推理引擎，在 可配置的循环模式 下执行目标检测推理。

```
cd $HOME/azurengine/yolov3-rt/build
./yolo-trt
```

默认情况下，Demo 仅执行 一次推理。用于压力测试时，需通过 `-l` 参数显式指定 循环次数。

8.2 基于循环的压力测试（推荐）

为启用 长时间压力测试，请设置较大的循环次数：

```
./yolo-trt -l100000
```

该命令将连续执行 **100,000** 次 **YOLOv3** 推理，用于验证：

- 长时间运行稳定性
- 资源泄漏情况
- 性能退化
- 功耗与温度一致性

注意：循环次数可根据测试时长灵活调整。对于过夜或多日测试，建议使用足够大的循环值。

8.3 命令行参数说明

显示帮助信息：

```
./yolo-trt -h
```

用法说明：

```
yolo demo usage:
   ./yolo-trt -mxx -lxx -dxx

Options:
   -m: 用于检测的输入图像路径
       使用 '-m<path>' (中间不加空格)
       若未指定，则从默认配置目录加载图像

   -l: 推理循环次数
       默认值: 1
       压力测试推荐: 较大数值 (如 10,000+)

   -d: 数据类型选择
       1: BF16
       默认: BF16 (唯一支持的模式)
```

8.4 示例命令

单张图片，**100** 次循环压力测试：

```
./yolo-trt -mconfigs/dog.jpg -l100 -d1
```

长时间可靠性测试（不指定图像）：

```
./yolo-trt -l50000 -d1
```

8.5 注意事项与最佳实践

- 该示例主要用于 稳定性验证，而非精度评估
- 在整个测试过程中保持 **RPP Server** 持续运行
- 无人值守测试建议：
  - 关闭 GUI 输出（如适用）
  - 使用 **AE-SMI** 监控功耗与温度
- 结合系统级监控工具，关注：

- CPU / 内存使用率
- PCIe 错误计数器
- 内核日志（dmesg）

以下为保持原有 **Markdown** 结构与格式不变的中文翻译：

9. 已知问题与故障排查

本节列出了在 Demo 执行过程中观察到的常见客户侧问题及其已验证的解决方案。

问题 1：LLM Demo 运行中

问题描述：**Graph Infer** 卡在 **prefilling** 阶段
根本原因：在执行 LLM Demo 时，RPP Server 仍处于运行状态。

解决方案：

在运行 LLM Demo 之前，确保 RPP Server 已被完全关闭：

```
# 确认不存在 rpp_server 进程
ps -ef | grep rpp_server

# 结束所有残留进程
sudo kill -9 <PID>

# 若问题仍然存在，重启系统
sudo reboot

# 重启后重新安装驱动并重试
cd $HOME/azurengine/rpp_drv
sudo insmod rpp.ko
sudo chmod 666 /dev/rpp0_entire_ctrl /dev/ve0_entire_ctrl
```

问题 2：OpenRT / CUDA 示例运行

问题描述：客户端连接服务器失败
根本原因：RPP Server 未运行或无法访问。

解决方案：

启动 RPP Server 并验证其连通性：

```
# 启动 RPP Server
cd /usr/local/rpp/bin
./rpp_server
```

```
# 在另一个终端中确认服务正在监听
netstat -tlnp | grep rpp_server

# 若仍然失败，请检查系统日志
dmesg | tail -50
journalctl -xe | tail -50
```

请确保在执行 OpenRT 和 CUDA Demo 期间，RPP Server 始终保持运行状态。

问题 3：OpenRT Python 示例运行

问题描述：运行 **OpenRT Python Demo** 时：

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python yolov8.py
```

报错如下：

```
ImportError: libpython3.11.so.1.0: cannot open shared object file: No such
file or directory
```

该错误发生在导入 OpenRT Python 运行时阶段：

```
import pyrt as trt
```

根本原因：

OpenRT Python 扩展模块：

```
/usr/local/rpp/lib/pyrt.cpython-311-x86_64-linux-gnu.so
```

是基于 **Python 3.11** 构建的，但系统动态链接器无法找到：

```
libpython3.11.so.1.0
```

该问题通常出现在 **Conda** 环境中运行 OpenRT Python Demo 的场景下。此时，`libpython` 位于 Conda 环境目录中，而未包含在系统默认的动态库搜索路径内。

解决方案：

- **1.** 定位 **`libpython3.11.so.1.0`**

确认 Python 共享库的位置：

```
locate libpython3.11.so.1.0
```

示例输出：

```
/home/azurengine/miniconda3/envs/az_samples_env/lib/libpython3.11.so.1.0
/home/azurengine/miniconda3/envs/xdl_llm/lib/libpython3.11.so.1.0
/home/azurengine/miniconda3/pkgs/python-3.11.14-h6fa692b_0/lib/
libpython3.11.so.1.0
```

- **2.** 将 **Conda** 库路径注册到动态链接器

将当前激活的 Conda 环境 `lib` 目录添加到动态链接器配置中：

```
sudo sh -c 'echo /home/azurengine/miniconda3/envs/az_samples_env/lib >> /etc/
ld.so.conf'
```

应用配置变更：

```
sudo /sbin/ldconfig
```

⚠️ 请确保该路径与当前运行 **Demo** 的 **Conda** 环境一致。

- **3.** 验证依赖关系是否正确解析（期望输出）

检查所有共享库依赖是否已被正确解析：

```
ldd -r /usr/local/rpp/lib/pyrt.cpython-311-x86_64-linux-gnu.so
```

期望输出（节选）：

```
linux-vdso.so.1 (0x00007ffef2be3000)
liburpp.so.2 => /usr/local/rpp/lib/liburpp.so.2
libnvonnxparser.so.6 => /usr/local/rpp/lib/libnvonnxparser.so.6
libpython3.11.so.1.0 => /home/azurengine/miniconda3/envs/az_samples_env/lib/
libpython3.11.so.1.0
libRppRT.so.1 => /usr/local/rpp/lib/libRppRT.so.1
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
/lib64/ld-linux-x86-64.so.2
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6
libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1
libcrypto.so.1.1 => /lib/x86_64-linux-gnu/libcrypto.so.1.1
librpp_asm.so => /usr/local/rpp/lib/librpp_asm.so
libjsoncpp.so.1 => /lib/x86_64-linux-gnu/libjsoncpp.so.1
```

如果 `libpython3.11.so.1.0` 成功解析到 Conda 环境路径（而非 `not found`），则说明问题已成功解决。

补充说明

- 该问题仅影响 **Python Demo**，不会影响 C++ Demo。
- 当切换 Conda 环境或 Python 版本时，可能需要相应更新动态链接器配置。
- 作为临时解决方案（不建议用于生产环境），可使用：

```
export LD_LIBRARY_PATH=/home/azurengine/miniconda3/envs/az_samples_env/lib:
$LD_LIBRARY_PATH
```

为了获得持久且系统级的正确性，推荐更新 `ld.so.conf.d` 并运行 `ldconfig`。

10. 结论

本文档详细介绍了在 x86_64 平台上 RPP Demo Guide 的安装、执行及故障排查 SOP。通过遵循文中步骤，客户可以成功搭建稳定的 RPP 环境，并通过多种 Demo 与 Sandbox 验证其功能。如在安装或 Demo 执行过程中遇到任何问题，请参考故障排查章节或联系 **XDL Technical Support**。

11. 致谢

本指南由 **XDL Technical Support** 团队基于大量客户交流与反馈编写完成。感谢所有为本安装流程的验证与改进做出贡献的客户与合作伙伴。

12. 反馈与贡献

我们欢迎任何有助于改进本文档的反馈与贡献。请将建议、修正或补充信息提交至 XDL Technical Support 或授权支持渠道。您的反馈将帮助我们持续提升文档质量与准确性。

13. 免责声明

本文档所包含的信息由 XDL Technologies 按“原样”提供。XDL Technologies 不对信息的准确性、完整性或可靠性作出任何明示或暗示的保证。XDL Technologies 保留在不事先通知的情况下修改或停止本文档中所述任何产品或服务的权利。

14. 版权声明

© 2026 XDL Technologies。保留所有权利。未经 XDL Technologies 事先书面许可，不得以任何形式或方式（包括电子、机械、复印、录音或信息存储与检索系统）复制或传播本文档的任何部分。

15. 技术支持

在客户交付相关项目中，请联系您指定的 **XDL Technical Support** 或授权支持渠道。问题反馈时，请务必包含以下信息：

- SDK 版本及创建时间戳

- 主机操作系统与内核版本
- RPP 硬件型号及 PCIe 拓扑结构
- 完整的 Demo 执行日志与错误信息
- 显示设备状态的 AE-SMI 输出

提供上述信息将显著加快问题定位与解决效率，并有助于提供更有针对性的技术支持。

如需进一步协助、性能调优、Demo 定制或平台相关指导，请联系 **XDL Technical Support** 或授权支持渠道。

16. 联系方式

如需进一步信息或技术支持，请联系：

- 官网：www.xdltek.cn
- 邮箱：
  - 技术支持：
    - sales@xdltek.com
    - support@xdltek.com

17. 法律声明

本文档仅供信息参考，不构成任何法律协议。XDL Technologies 对文档中的任何错误或遗漏不承担责任。本文档所包含的信息可能在不另行通知的情况下发生变更。

18. 商标声明

XDL 及 XDL 标识为 XDL Technologies 的商标。本文中提及的其他产品名称、公司名称、商标及注册商标均为其各自所有者的财产。

19. 注释

1. 所有规格与功能如有变更，恕不另行通知。
2. 性能指标基于内部测试结果，可能因系统配置与工作负载不同而有所差异。
3. 最新信息请参考 XDL Technologies 官方网站。

20. 安全与合规

XDL Technologies 产品符合相关安全与环保法规。用户应按照产品文档中所述的正确操作与使用规范，以确保安全并符合当地法规要求。

21. 发布说明

有关最新更新、缺陷修复及功能增强，请参考 XDL Technologies 随每次 SDK 发布提供的官方 Release Notes。

修订记录

版本	日期	作者	描述
2.0	2026-01-18	XDL Technical Support Team	初始版本
2.1	2026-02-07	XDL Technical Support Team	发布版